Website, Store & Store View in Magento 2

A from-zero explanation, the real-world reasons they exist,
and a full map of everything you built in **Codilar_Twoxl**

Project: 2XL Home · Module: `Codilar_Twoxl` · Theme: `Codilar/twoxl`
Generated 30 October 2026

What is inside this guide

1. **Part 1 — The idea.** Why Magento splits a shop into three levels, explained with no Magento jargon.
2. **Part 2 — The four scopes.** Global, Website, Store (Group), Store View — and exactly what each one owns.
3. **Part 3 — The database behind it.** The four tables that make all of this real.
4. **Part 4 — Real-world scenarios.** Eight situations from actual projects and which level solves each one.
5. **Part 5 — How Magento knows which store you are in.** URLs, store codes, base URLs.
6. **Part 6 — Configuration inheritance.** The fallback chain, and the "Use Default" checkbox explained properly.
7. **Part 7 — Attribute scope.** Why a price can have a different value per website but a SKU cannot.
8. **Part 8 — YOUR CODE, file by file.** Every file in your project that touches websites/stores/store views, what it does and why.
9. **Part 9 — The traps you already hit.** Real bugs from your own patches, and what they teach.
10. **Part 10 — Cheat sheet, commands and practice exercises.**

Part 1 — The idea, with zero jargon

Forget Magento for two minutes. Imagine you own a furniture business called **2XL Home**.

The business grows in three different directions

Direction 1 — new country. You open in India. Indian customers pay in rupees, not dirhams. An Indian customer account is a different account from a UAE one. The legal entity, the tax rules, the delivery promises are all different. This is a **separate business unit**, even though the brand is the same.

Direction 2 — new kind of shop. In the UAE you already sell furniture; now you want a separate outlet that sells only office furniture, with its own product range and its own homepage. Same company, same currency, same customers — but a **different catalogue**.

Direction 3 — new language. Your UAE shop needs an Arabic version. Same products, same prices, same stock, same customer accounts, same cart. Only the **words on screen** change.

> **Those three directions are exactly Magento's three levels.**
> Direction 1 = a **Website**. Direction 2 = a **Store** (also called a Store Group). Direction 3 = a **Store View**.

The shopping-mall analogy

```
INSTALLATION  (one Magento codebase, one database, one admin panel)
│
├── WEBSITE "2XL Home UAE"          ← the legal/commercial entity
│   │   owns: currency (AED), customer accounts, the cart, payment & shipping
│   │
│   └── STORE GROUP "UAE Store"     ← the catalogue / the aisles
│       │   owns: which root category (= which products & menu) is sold
│       │
│       ├── STORE VIEW "ae_en"      ← the language / the presentation
│       └── STORE VIEW "ae_ar"      ← same products, Arabic words
│
└── WEBSITE "2XL Home India"        ← a second entity: INR, its own customers
    │
    └── STORE GROUP "India Store"
        │
        └── STORE VIEW "in_en"
```

- A **Website** is the *shop as a business*. Money, accounts and the cart live here.
- A **Store Group** is the *shelves*. It decides which products exist, by pointing at one root category.
- A **Store View** is the *signage*. Language, wording, colours, CMS pages, translated product names.

The single most useful sentence in this whole document

> **If two shops must have different money or different customer logins → you need two Websites.**
> **If they only need different products → one Website, two Store Groups.**
> **If they only need different words → one Store Group, two Store Views.**

Your project is the first case. UAE sells in AED, India sells in INR, and `rahul.menon@example.com` registering in India must not automatically be logged in on the UAE site. So you created a second **Website** — and because a website cannot exist without a group and a view underneath it, you created all three in one go.

Why a Store Group sits in the middle (the level everyone forgets)

Beginners always ask why there are three levels and not two. The reason is the **root category**.

Every store group points at exactly one root category. Everything underneath that root is "the catalogue and the menu" for that group. Two groups under the same website = two different product ranges, sharing one currency, one cart and one set of customers.

In your code this is the line that does it, in `app/code/Codilar/Twoxl/Setup/Patch/Data/AddIndiaWebsite.php`:

```
/** Shared with the UAE storefront - one catalogue, two regions. */
private const ROOT_CATEGORY_ID = 2;

$group->setWebsiteId((int)$website->getId())
    ->setCode(self::GROUP_CODE)          // 'in_store'
    ->setName('India Store')
    ->setRootCategoryId(self::ROOT_CATEGORY_ID);   // ← 2 = the SAME root as UAE
```

You deliberately pointed India at root category **2**, the same root the UAE website uses. That is why both countries show the identical product tree and the identical top menu. If you had created root category 5 and pointed India there, India would have had a completely separate catalogue and menu — same code, same database, totally different shop.

> **A very common real-world mistake:** someone wants "the same products in two countries" and creates a second root category by copy-pasting the first. Now every new product has to be added to two trees forever. Sharing one root category — what you did — is almost always correct.

Part 2 — The four scopes and what each one owns

Magento calls these levels **scopes**. There are four, because above everything sits **Default** (also called Global), the value used when nobody has overridden anything.

Scope	What it represents	What is decided at this level
Default (Global)	The whole installation. One row, `scope_id = 0`.	Anything that must be true everywhere: SKU, product type, attribute set, the "Catalog Price Scope" switch itself, "Add Store Code to URLs", cron, indexers, security settings.
Website	A commercial entity. Your `base` (UAE) and `in` (India).	**Base currency**, customer accounts (who can log in where), the shopping cart & quote, which products are *assigned* and sellable, payment methods, shipping methods, tax, stock when MSI is per-website, price & special price *if* price scope is set to Website.
Store (Store Group)	A catalogue. Your `in_store` and the existing UAE group.	Exactly one thing that matters: the **root category** — i.e. which products and which navigation menu this branch sells. It has no configuration page of its own.
Store View	A presentation. Your `ae_en` and `in_en`.	Locale/language, translations, theme, CMS pages & blocks, product *name/description/status/visibility*, display currency, URL rewrites, design, and all your `twoxl/region/*` wording.

Rules you can rely on

- A Website contains one or more Store Groups. A Store Group contains one or more Store Views.
- **Nothing can be empty.** A website needs a default group; a group needs a default store view. That is why your patch sets the defaults *last*, after all three rows exist:

  ```
  // Defaults can only be pointed at rows that exist, so they are set last.
  $website->setDefaultGroupId((int)$group->getId());
  $this->websiteResource->save($website);

  $group->setDefaultStoreId((int)$store->getId());
  $this->groupResource->save($group);
  ```

- Store View IDs start at **1**. ID **0** is the reserved "admin" store view — the one the backend itself runs in, and the one that holds default attribute values.
- A customer belongs to a **website**, never to a store view. (It is stored as `website_id` on the customer row.)
- An order belongs to a **store view** (so you know which language/region it came from) but its totals follow the website's currency.

> **Where you can see this in your own admin:** *Stores → All Stores* shows the three-column tree (Website / Store / Store View). *Stores → Configuration* has the **scope switcher** in the top-left — that dropdown is literally "which of these four scopes am I editing right now".

Part 3 — The database behind it

All of this is just four tables. Once you can picture them, nothing about scopes is mysterious again.

1. `store_website`

website_id	code	name	sort_order	default_group_id	is_default
0	admin	Admin	0	0	0
1	base	Main Website	0	1	1
2	in	2XL Home India	2	2	0

Row 3 is the one your `AddIndiaWebsite` patch inserts. `is_default = 0` because the UAE site stays the installation default.

2. `store_group`

group_id	website_id	code	name	root_category_id	default_store_id
0	0	default	Default	0	0
1	1	main_website_store	Main Website Store	2	1
2	2	in_store	India Store	**2**	2

Look at `root_category_id`: both groups say **2**. That single shared number is why India and the UAE sell the identical catalogue.

3. `store` (this table holds *store views*, despite the name)

store_id	code	website_id	group_id	name	is_active
0	admin	0	0	Admin	1
1	**ae_en**	1	1	UAE English	1
2	**in_en**	2	2	India English	1

> **The naming trap that confuses everyone:** the table called `store` holds **store views**, and the table called `store_group` holds what the admin UI calls a **Store**. In PHP, `StoreManagerInterface::getStore()` returns a **store view**, and `getGroup()` returns the middle level. Whenever you read `$storeId` in Magento code, read it as "store *view* id".

Store view 1 shipped as code `default`. Your patch renames it, and the comment in the file explains exactly why:

```
/** code => name for the view that shipped as "default" */
private const RENAMED_VIEW = ['default' => ['ae_en', 'UAE English']];
```

Because `web/url/use_store` gets switched on (next section), every store view's code appears in its URLs. Left alone the UAE site would have answered on `/default/`, which looks broken to a shopper. Renaming it to `ae_en` gives you a tidy `/ae_en/` and `/in_en/` pair.

4. `core_config_data` — the table that makes inheritance work

config_id	scope	scope_id	path	value
…	default	0	currency/options/base	AED
…	default	0	catalog/price/scope	1
…	default	0	web/url/use_store	1
…	websites	2	currency/options/base	INR
…	websites	2	general/country/default	IN
…	websites	1	general/country/default	AE
…	stores	2	general/locale/code	en_IN
…	stores	2	twoxl/region/headline	Now delivering across India

Three columns do all the work: **scope** (`default` / `websites` / `stores`), **scope_id** (which website or which store view), and **path** (the `section/group/field` string).

> **Rows only exist where someone overrode something.** There is no `twoxl/region/headline` row for the UAE — the UAE reads the value from `etc/config.xml` . That is the exact point your `ConfigureRegionBanner` patch comment makes: *"one storefront inherits, the other overrides, and only the overridden rows exist in core_config_data."*

Part 4 — Real-world scenarios: which level do I use?

This is the part you will actually use on client projects. For each situation: what the client asks for, and what you build.

Scenario 1 — "We're launching in a second country"

Client says: "We sell in Dubai, now we want India too. Rupee prices, Indian delivery message, same furniture."

You build: a new **Website** → one Store Group pointing at the *same* root category → one Store View.

Why website: different base currency, and Indian customers must have their own accounts. Both of those are website-level facts.

This is literally your project. Files: `AddIndiaWebsite.php`, `ConfigureIndiaStore.php`.

Scenario 2 — "We need an Arabic version of the UAE site"

Client says: "Same products, same prices, same cart — just in Arabic, right-to-left."

You build: one extra **Store View** under the existing UAE group. Set `general/locale/code = ar_SA` at *store view* scope, assign an RTL theme at store view scope, translate CMS blocks per store view.

Why not a website: the shopper must be able to switch language and keep their cart. A cart belongs to a website — switch website and the cart is gone.

Your region strip and switcher are already built for this: both read at store-view scope, so an Arabic view would pick up Arabic wording with no code change.

Scenario 3 — "Separate B2B and B2C shops"

Client says: "Retail customers see one price, trade customers log in and see another, with different products."

You build: two **Websites** — because B2B needs its own customer accounts, its own payment methods (invoice/credit terms) and often its own prices. Each gets its own store group with its own root category if the ranges differ.

Counter-example: if the only difference is price, customer groups + tier prices on **one** website is simpler. Reach for a second website only when logins, currency or payment genuinely differ.

Scenario 4 — "Two brands, one warehouse"

Client says: "We own 'ModernLiving' and 'ClassicWood'. Different domains, different logos, shared stock and one admin."

You build: two **Websites**, each with its own base URL and its own theme. One admin panel, one product database, products assigned to one or both websites.

The assignment step is what your `SetIndiaCatalogPrices` patch does:

```
$this->productAction->updateWebsites(array_values($ids), [$websiteId], 'add');
```

A product that is not assigned to a website simply does not exist on that storefront — no 404 handling needed, it is invisible.

Scenario 5 — "An outlet / clearance shop"

Client says: "Same company, same customers, same currency — but a separate site selling only ex-display items."

You build: a second **Store Group** under the existing website, pointing at a new root category "Outlet". Customers stay logged in across both, the cart carries over, and only the product tree changes. This is the textbook case for the middle level.

Scenario 6 — "Different wording per region, same code"

Client says: "The UAE banner should say free installation; India should say 40+ cities, in green."

You build: config fields with `showInStore="1"`, defaults in `etc/config.xml`, and per-store-view overrides. **No second template.**

This is your `twoxl/region/*` section exactly — `system.xml` + `config.xml` + `RegionBanner.php` + `region-strip.phtml`.

Scenario 7 — "Turn the enquiry form off in one country only"

Client says: "Stop taking quote requests in India while we hire the team there."

You build: nothing. Someone goes to *Stores → Configuration*, switches scope to **India English**, unticks "Use Website", sets Enable = No, saves. One row appears in `core_config_data`.

That works because every field in your `twoxl_enquiry` section carries `showInStore="1"`, and because `EnquiryConfig` reads with `ScopeInterface::SCOPE_STORE`. Design the config right once and the client never needs you again for this kind of request.

Scenario 8 — "One product, two prices"

Client says: "The sofa is AED 13,999 in Dubai and ₹319,999 in India."

You build: set **Catalog Price Scope = Website**, then write each price at the right scope. Your `ConfigureIndiaStore` does the first half:

```
// 1 = website; each website then owns its base currency and its own prices.
'catalog/price/scope' => '1',
```

and `RestoreRegionalPrices` does the second half. Part 9 explains the bug that made that second patch necessary — it is the most valuable lesson in your whole codebase.

Decision shortcut to memorise

Different currency or different logins → **Website**.

Different product range or menu → **Store Group**.

Different language, wording, theme or CMS → **Store View**.

Different price for the same product → one of the first two, plus the price-scope switch.

Part 5 — How Magento knows which store view you are in

On every request Magento must resolve exactly one store view before it can render anything. There are four ways it can happen.

1. Store code in the URL path (what your project uses)

```
https://2xl.local/ae_en/sofas/niecel-sectional
https://2xl.local/in_en/sofas/niecel-sectional
```

Switched on by `web/url/use_store = 1`. Your `ConfigureIndiaStore` sets it, and the class comment explains a subtlety worth remembering:

```
/**
 * "Add Store Code to Urls" is global rather than per website (showInDefault only, and
 * Store::isUseStoreInUrl reads the default scope), so switching it on here is what puts
both
 * /ae_en/ and /in_en/ in front of storefront URLs.
 */
```

It is all-or-nothing for the installation. You cannot give India a prefix and leave the UAE without one — which is precisely why the `default` view had to be renamed.

2. A different base URL per website

The production approach for separate brands or countries: set `web/unsecure/base_url` and `web/secure/base_url` at **website** scope.

```
website 'base' →  https://2xlhome.ae/
website 'in'   →  https://2xlhome.in/
```

Then your web server passes `MAGE_RUN_TYPE=website` and `MAGE_RUN_CODE=in` for that vhost, and no store code appears in the URL at all. Apache example:

```
SetEnvIf Host ^2xlhome\.in$ MAGE_RUN_CODE=in
SetEnvIf Host ^2xlhome\.in$ MAGE_RUN_TYPE=website
```

3. The `___store` query parameter

`?___store=in_en` forces a store view and sets a cookie. This is what Magento's own built-in language switcher uses.

4. Nothing matched → the default

Falls back to the default website → its default group → its default store view. This is why those "default" pointers must always be valid.

Your region switcher takes the base-URL route

`app/code/Codilar/Twoxl/ViewModel/RegionSwitcher.php`

```
'url' => (string)$store->getBaseUrl(UrlInterface::URL_TYPE_LINK),
```

Asking each store view for its own base URL is the robust choice: today that returns `/ae_en/` and `/in_en/`; the day the client buys `2xlhome.in` and you set a website-scope base URL, the same line starts returning the real domain. No template change, no code change.

Part 6 — Configuration inheritance, properly explained

When code asks for a config value at store view 2, Magento walks **up** until it finds a row:

```
ask for: twoxl/region/headline   at store view 'in_en' (store_id 2)

1. core_config_data  scope='stores'    scope_id=2   → FOUND "Now delivering across India"  ✓ stop
2. core_config_data  scope='websites'  scope_id=2   → (not reached)
3. core_config_data  scope='default'   scope_id=0   → (not reached)
4. etc/config.xml    <default>                      → (not reached)


ask for: twoxl/region/headline   at store view 'ae_en' (store_id 1)

1. core_config_data  scope='stores'    scope_id=1   → nothing
2. core_config_data  scope='websites'  scope_id=1   → nothing
3. core_config_data  scope='default'   scope_id=0   → nothing
4. etc/config.xml    <default>                      → FOUND "Delivering across the UAE"  ✓
```

That is the entire mechanism. Four steps, first match wins.

The three places a default can live

1. `etc/config.xml` **in your module** — the shipped default. No database row. This is where your UAE wording lives.
2. `core_config_data` **with** `scope='default'` — an installation-wide override a human or a patch saved.
3. `app/etc/env.php` **under** `system` — a locked value. It wins over everything and greys the field out in the admin. Used for per-environment secrets.

What the "Use Default" / "Use Website" checkbox really does

Ticking it **deletes the row** for that scope. Unticking it and saving **inserts a row**. That is all. Nothing is copied — the value you then see is simply whatever the fallback chain produces.

Writing config from code — the three scopes

Your patches show all three in one file,

`app/code/Codilar/Twoxl/Setup/Patch/Data/ConfigureIndiaStore.php`:

```
// DEFAULT scope — the whole installation
$this->configWriter->save($path, $value, ScopeConfigInterface::SCOPE_TYPE_DEFAULT, 0);

// WEBSITE scope — India only; the UAE keeps the AED defaults
$websiteId = (int)$this->storeManager->getWebsite('in')->getId();
$this->configWriter->save($path, $value, ScopeInterface::SCOPE_WEBSITES, $websiteId);

// STORE VIEW scope
$storeId = (int)$this->storeManager->getStore('in_en')->getId();
$this->configWriter->save($path, $value, ScopeInterface::SCOPE_STORES, $storeId);
```

And the split of settings in that file is a good model to copy on any multi-site project:

Scope	Setting	Why there
default	`catalog/price/scope = 1`	Magento only reads this flag globally; it cannot differ per website.
default	`web/url/use_store = 1`	Same — the field is `showInDefault` only.
websites (in)	`currency/options/base = INR`	Base currency is a website-level fact. This is the whole reason India is a website.
websites (in)	`general/country/default = IN`	Drives tax, shipping origin, and the label in your region switcher.
stores (in_en)	`general/locale/code = en_IN`	Language and number/date formatting are presentation → store view.

Reading config at the right scope

`app/code/Codilar/Twoxl/ViewModel/RegionBanner.php`

```
return trim((string)$this->scopeConfig->getValue($path, ScopeInterface::SCOPE_STORE));
```

Passing `SCOPE_STORE` with no id means "the store view of the current request". On the storefront that is what you almost always want.

> **The most common bug in Magento config code:** calling `$scopeConfig->getValue('my/path')` with no scope argument. That reads the **default** scope and silently ignores every per-store override. The site looks fine on the main store view and wrong on every other one. Your code never does this — every read in `RegionBanner` and `EnquiryConfig` passes a scope.

Passing an explicit store id — and why your notifier must

There is one case where "current store view" is wrong: background work.

`app/code/Codilar/Twoxl/Model/EnquiryNotifier.php`

```
$storeId = (int)$enquiry->getStoreId();

if (!$this->config->isNotificationEnabled($storeId)) {
    return;
}
$recipients = $this->config->getRecipients($storeId);
```

The enquiry remembers which store view it came from, so the email is sent with that store's settings, that store's template and that store's recipients — even if the mail is later re-sent from a cron job or the admin, where "the current store view" would be the admin store (id 0). This is why `EnquiryConfig` accepts `?int $storeId = null` on every method.

Part 7 — Attribute scope (why price can vary but SKU cannot)

Config is one half of the story. The other half is **product attribute values**, which have their own three-level scope: **Global**, **Website**, **Store View**.

Attribute scope	Examples	Stored how
Global	`sku`, `weight`, `color`, `tax_class_id`, attribute-set membership	One row, `store_id = 0`. Same everywhere, always.
Website	`price`, `special_price`, `cost` — *only when* Catalog Price Scope = Website	A row per website's store views, written through the website.
Store View	`name`, `description`, `status`, `visibility`, `url_key`, `meta_title`	A row per store view, falling back to `store_id = 0`.

EAV values live in tables like `catalog_product_entity_decimal`, with columns `(attribute_id, store_id, entity_id, value)`. The fallback is the same idea as config: **look for a row at this store view; if none, use the row at store_id 0.**

```
catalog_product_entity_decimal   (attribute 'price')

 entity_id | store_id | value      meaning
-----------+----------+--------    ----------------------------------------
    42     |    0     |  13999  ←  the default row. UAE reads this.
    42     |    2     | 319999  ←  override for store view 2 (in_en). India reads this.
```

The price-scope switch is a runtime re-scope, not a migration

Your `ConfigureIndiaStore` comment says it precisely:

```
/**
 * Catalog price scope moves to Website, which is what lets each website carry its own base
 * currency - Catalog\Model\Product\Attribute\Backend\Price::setScope() reads that flag and
 * re-scopes the price attribute at runtime, so nothing in eav has to be rewritten.
 */
```

Nothing is migrated when you flip the switch. The `price` attribute simply *reports* a different scope from that moment on, and every read and write starts respecting store/website rows. Keep this in mind — it is the root of the bug in Part 9.

Writing a scoped attribute value

`app/code/Codilar/Twoxl/Setup/Patch/Data/RestoreRegionalPrices.php`

```
$this->productAction->updateAttributes(
    [$id],
    ['price' => $price, 'special_price' => $specialPrice],
    $storeId          // 0 = the default row; 2 = the India store view
);
```

Passing `0` writes the fallback row that the UAE inherits. Passing the India store id writes an override. And the comment in `SetIndiaCatalogPrices` explains why a store-view write is acceptable here:

```
// India has a single view, so a store level write is the website value.
```

If India later gains a Hindi view, that assumption breaks and the prices would need to be written at website scope instead. Worth a note in your project backlog.

Part 8 — Your code, file by file

Everything below is a file in **your** project that touches websites, stores or store views. Read this section with the files open beside it.

8.0 The map at a glance

File	Scope it works at	What it does
`Setup/Patch/Data/ConfigureStore.php`	default	Baseline AED currency, URL suffixes, SEO titles
`Setup/Patch/Data/AddIndiaWebsite.php`	creates all 3 levels	Website + Group + Store View for India; renames `default` → `ae_en`
`Setup/Patch/Data/ConfigureIndiaStore.php`	default + website + store	Price scope, store code in URLs, INR, country IN, locale en_IN
`Setup/Patch/Data/ConfigureUaeRegion.php`	website	Names the UAE's country so the switcher has a label
`Setup/Patch/Data/ConfigureRegionBanner.php`	store view	India-only banner wording and green accent
`Setup/Patch/Data/SetIndiaCatalogPrices.php`	website + store view	Assigns products to the India website, writes INR prices
`Setup/Patch/Data/RestoreRegionalPrices.php`	store view 0 + store view 2	Fixes prices that landed on the wrong row
`Setup/Patch/Data/GenerateRegionalUrlRewrites.php`	all store views	Builds catalogue URL rewrites for the new view
`Setup/Patch/Data/SeedCustomers.php`	website + store view	One demo customer per website
`Setup/Patch/Data/SeedProductEnquiries.php`	store view	Demo enquiries tagged with their origin store
`etc/config.xml`	default	Shipped defaults for region strip + enquiry form
`etc/adminhtml/system.xml`	declares scopes	`showInDefault/Website/Store` on every field
`etc/db_schema.xml`	store view	`store_id` column + FK on the enquiry table
`ViewModel/RegionBanner.php`	reads store view	Banner text, subline, currency badge, accent colour
`ViewModel/RegionSwitcher.php`	reads all store views	Builds the globe-icon country menu
`ViewModel/PriceFormatter.php`	reads store view	AED glyph vs normal currency formatting
`ViewModel/Pdp.php`	reads store view/group	Base URL + root category for breadcrumbs

`Model/Config/EnquiryConfig.php`	reads store view	All enquiry settings, with optional explicit store id
`Controller/Enquiry/Save.php`	writes store view	Stamps the enquiry with the store it came from
`Model/EnquiryNotifier.php`	explicit store view	Sends the mail in that store's context
`Model/GuestWishlist.php`	store view	Loads products in the current store's context
`view/adminhtml/ui_component/...enquiry_listing.xml`	store view	Store View grid column + filter
`theme .../Magento_Theme/layout/default.xml`	wires view models	Injects RegionBanner and RegionSwitcher into blocks
`theme .../header/region-strip.phtml`	renders store view data	The coloured strip under the header
`theme .../header/account.phtml`	renders store list	The globe country dropdown

8.1 Creating the three levels — `AddIndiaWebsite.php`

`app/code/Codilar/Twoxl/Setup/Patch/Data/AddIndiaWebsite.php`

This is the single most important file for understanding the hierarchy, because it builds all three levels by hand, in the only order that works.

Step 1 — rename the shipped view (so the UAE does not end up on `/default/`):

```
private const RENAMED_VIEW = ['default' => ['ae_en', 'UAE English']];
```

Step 2 — the Website. Note the `load($website, 'in', 'code')` pattern: load by code, and only insert if nothing came back. That is what makes a data patch safe to run on an install where it has partly run before.

```
$website = $this->websiteFactory->create();
$this->websiteResource->load($website, self::WEBSITE_CODE, 'code');
if (!$website->getId()) {
    $website->setCode(self::WEBSITE_CODE)      // 'in'
        ->setName('2XL Home India')
        ->setSortOrder(2)
        ->setIsDefault(false);                 // UAE stays the installation default
    $this->websiteResource->save($website);
}
```

Step 3 — the Store Group, attached to the website and pointing at the shared root category:

```
$group->setWebsiteId((int)$website->getId())
    ->setCode(self::GROUP_CODE)                // 'in_store'
    ->setName('India Store')
    ->setRootCategoryId(self::ROOT_CATEGORY_ID);   // 2 — same catalogue as UAE
```

Step 4 — the Store View, which needs *both* parent ids:

```
$store->setCode(self::STORE_CODE)              // 'in_en'
    ->setName('India English')
    ->setWebsiteId((int)$website->getId())     // ← denormalised on purpose
    ->setGroupId((int)$group->getId())
    ->setSortOrder(0)
    ->setIsActive(1);                          // inactive views are invisible on the
storefront
```

> A store view stores **both** `website_id` and `group_id`, even though the website could be derived from the group. Magento denormalises it so that resolving a request never needs an extra join. Keep them consistent if you ever write to this table directly.

Step 5 — the default pointers, last. Already shown in Part 2 — a website cannot point at a group that does not exist yet.

Step 6 — ordering. Data patches do not run in file order; they run in dependency order:

```
public static function getDependencies(): array
{
    return [ConfigureStore::class];
}
```

Your whole chain is: `ConfigureStore → AddIndiaWebsite → ConfigureIndiaStore → SetIndiaCatalogPrices → RestoreRegionalPrices`, with `ConfigureUaeRegion`, `ConfigureRegionBanner`, `SeedCustomers` and `SeedProductEnquiries` hanging off `AddIndiaWebsite`, and `GenerateRegionalUrlRewrites` waiting for the prices. Every one of those arrows exists for a reason: you cannot configure a website before it exists, and you cannot price a product for a website that has not been created.

8.2 Declaring config that can vary — `system.xml`

`app/code/Codilar/Twoxl/etc/adminhtml/system.xml`

Three attributes on every field and group decide which scopes the admin can edit:

```
<field id="headline" translate="label" type="text" sortOrder="20"
       showInDefault="1" showInWebsite="1" showInStore="1" canRestore="1">
```

Attribute	Effect
`showInDefault="1"`	Editable in the Default Config scope.
`showInWebsite="1"`	Also editable when the scope switcher is on a website. Adds the "Use Default" checkbox.
`showInStore="1"`	Also editable per store view. Adds the "Use Website" checkbox.
`canRestore="1"`	Shows "Use Default" even in the default scope, restoring the `config.xml` value.

Your file's header comment states the design intent in one line:

```
Every field is showInStore="1", which is what puts the scope switcher on the page and lets
one storefront say something different from the other without a code change.
```

> **Rule of thumb when you write a new module:** if a value is *content* (text, colour, on/off for a feature), make it `showInStore="1"`. If it is *commercial* (an API key for a payment gateway, a warehouse address), `showInWebsite="1"` is usually the right ceiling. If it is *technical* (a cron expression, an index mode), leave it `showInDefault="1"` only — and remember Magento may refuse to read it at any other scope anyway.

8.3 Shipping the defaults — `config.xml`

`app/code/Codilar/Twoxl/etc/config.xml`

```
<default>
    <twoxl>
        <region>
            <enabled>1</enabled>
            <headline>Delivering across the UAE</headline>
            <subline>Free delivery &amp;amp; installation on every order</subline>
            <accent>#a8731a</accent>
        </region>
    </twoxl>
</default>
```

The XML path maps one-to-one onto the config path: `<twoxl><region><headline>` is `twoxl/region/headline`. The `<default>` wrapper means "scope = default".

Then `ConfigureRegionBanner.php` overrides four of them for **one store view only**:

```
private const SETTINGS = [
    'twoxl/region/enabled'  => '1',
    'twoxl/region/headline' => 'Now delivering across India',
    'twoxl/region/subline'  => 'Free delivery & installation in 40+ cities',
    'twoxl/region/accent'   => '#1b7a4b',
];
...
$this->configWriter->save($path, $value, ScopeInterface::SCOPE_STORES, $storeId);
```

Open *Stores → Configuration → General → 2XL Home → Region Strip* and flip the scope switcher between the two store views. On India the checkboxes are unticked and the fields are editable; on the UAE they are ticked and greyed. That visual difference *is* the four rows in `core_config_data`.

8.4 Reading it back — `RegionBanner.php`

`app/code/Codilar/Twoxl/ViewModel/RegionBanner.php`

```
public function isEnabled(): bool
{
    return $this->scopeConfig->isSetFlag(self::PATH_ENABLED, ScopeInterface::SCOPE_STORE);
}

public function getCurrencyCode(): string
{
    return (string)$this->storeManager->getStore()->getCurrentCurrencyCode();
}
```

Two different store-view-aware APIs in one class:

- `scopeConfig->getValue(path, SCOPE_STORE)` — for anything from `core_config_data` / `config.xml`.
- `storeManager->getStore()` — for facts about the store view object itself: id, code, name, base URL, root category, currency.

`getCurrentCurrencyCode()` is worth noticing. It returns the **display** currency (what the shopper chose), which falls back to the website's default currency. That is why the badge on the strip reads AED on one site and INR on the other with no conditional anywhere.

And the accent colour is validated before it reaches a `<style>` block, because admin-editable values are untrusted input:

```
return preg_match('/^#[0-9a-fA-F]{6}$/', $accent) ? $accent : self::FALLBACK_ACCENT;
```

8.5 Listing every store view — `RegionSwitcher.php`

`app/code/Codilar/Twoxl/ViewModel/RegionSwitcher.php`

```
foreach ($this->storeManager->getStores() as $store) {
    if (!$store->getIsActive()) {
        continue;
    }

    $regions[] = [
        'label'     => $this->countryLabel($store->getId(), (string)$store->getName()),
        'currency'  => (string)$store->getCurrentCurrencyCode(),
        'url'       => (string)$store->getBaseUrl(UrlInterface::URL_TYPE_LINK),
        'isCurrent' => (int)$store->getId() === $currentId,
    ];
}
```

Four things here are worth copying into every multi-store project you ever build:

1. **The list is derived, not hard-coded.** Add a store view in the admin and it appears in the header. The class comment says exactly this: *"Built from the active store views rather than a fixed list, so adding a region in the admin is enough for it to appear."*
2. `getStores()` **excludes the admin store (id 0) by default**, so you never accidentally offer shoppers a link to the backend store view.
3. **Inactive views are skipped.** Disabling a store view in the admin is enough to pull a country out of the menu.

4. **The label comes from config at that store's scope**, not from a PHP array:

```
$code = (string)$this->scopeConfig->getValue(
    'general/country/default', ScopeInterface::SCOPE_STORE, $storeId
);
$name = (string)$this->countryFactory->create()->loadByCode($code)->getName();
```

   Note the **explicit** `$storeId`. The loop is reading settings for store views other than the current one, so "current scope" would give the same answer every iteration. This is the classic reason to pass an explicit id.

And this is where `ConfigureUaeRegion.php` earns its place — the UAE website had no `general/country/default` at all, so the switcher had nothing to label it with:

```
/**
 * Names the country of the original storefront.
 *
 * It was never set, which left the header's region list without a label for the UAE once
 * Codilar\Twoxl\ViewModel\RegionSwitcher started building that list from the stores.
 */
```

A small patch, but a good lesson: **when you make something data-driven, go back and fill in the data the old hard-coded version never needed.**

8.6 Currency-aware presentation — `PriceFormatter.php`

`app/code/Codilar/Twoxl/ViewModel/PriceFormatter.php`

```
public function isAed(): bool
{
    return $this->getCurrencyCode() === self::AED;
}

public function money(float $amount, bool $forceDecimals = false): string
{
    if ($this->isAed()) {
        $number = $forceDecimals ? $this->formatDecimal($amount) : $this-
>formatNumber($amount);
        return self::GLYPH . htmlspecialchars($number, ENT_QUOTES, 'UTF-8');
    }

    return htmlspecialchars((string)$this->priceCurrency->format($amount, false),
ENT_QUOTES, 'UTF-8');
}
```

This is a nice example of **branching on store context instead of duplicating a template**. The UAE gets the custom dirham glyph; every other region falls through to Magento's own `PriceCurrencyInterface::format()`, which already knows the store's currency symbol, decimal separator and thousands separator from the locale. India therefore renders `₹3,19,999.00` in the Indian grouping style with no India-specific code.

The `formatNumber()` method also shows a real-world detail: dropping `.00` on whole amounts is a 2XL design decision, not a Magento one, so it belongs in your view model rather than in a template.

8.7 Store-view data in your database table — `db_schema.xml`

`app/code/Codilar/Twoxl/etc/db_schema.xml`

```
<!-- Nullable so an enquiry outlives the store view it came from. -->
<column xsi:type="smallint" name="store_id" unsigned="true" nullable="true"
identity="false"
        comment="Store view the enquiry was raised in"/>

<constraint xsi:type="foreign"
referenceId="CODILAR_TWOXL_PRODUCT_ENQUIRY_STORE_ID_STORE_STORE_ID"
            table="codilar_twoxl_product_enquiry" column="store_id" referenceTable="store"
            referenceColumn="store_id" onDelete="SET NULL"/>
```

This is the pattern for every custom entity you will ever build. Three decisions, all correct here:

- **Column type** `smallint unsigned` — matches `store.store_id` exactly, which a foreign key requires.
- `nullable="true"` — so the row survives if the store view is deleted.
- `onDelete="SET NULL"` — not `CASCADE`. Deleting a store view must never silently delete a year of customer enquiries. Magento's own `sales_order` table makes the same choice.

The value is stamped on at write time, in `app/code/Codilar/Twoxl/Controller/Enquiry/Save.php`:

```
$enquiry->addData($data + [
    'status'   => ProductEnquiry::STATUS_NEW,
    'store_id' => (int)$this->storeManager->getStore()->getId(),
]);
```

And read back in the admin grid,

`app/code/Codilar/Twoxl/view/adminhtml/ui_component/codilar_twoxl_enquiry_listing.xml`:

```
<column name="store_id" class="Magento\Store\Ui\Component\Listing\Column\Store"
sortOrder="70">
    <settings>
        <label translate="true">Store View</label>
        <bodyTmpl>ui/grid/cells/html</bodyTmpl>
        <sortable>false</sortable>
    </settings>
</column>
```

`Magento\Store\Ui\Component\Listing\Column\Store` is Magento's own column class — it renders the full *Website / Store / Store View* breadcrumb instead of a raw number. Reuse it; never render `store_id` as an integer in a grid.

Your grid adds one refinement most developers miss, and the comment explains it:

```
<!--
    The Store View column renders HTML, so the export cannot read labels off it. A select
    filter gives the grid a Store View dropdown and, because the exporter reads its
    options, makes the column export as store names instead of raw ids.
-->
<filterSelect name="store_id" provider="${ $.parentName }">
    <settings>
        <captionValue>0</captionValue>
        <options class="Magento\Store\Ui\Component\Listing\Column\Store\Options"/>
        ...
```

8.8 Store-aware product loading — `GuestWishlist.php`

`app/code/Codilar/Twoxl/Model/GuestWishlist.php`

```
$storeId = (int)$this->storeManager->getStore()->getId();
...
$product = $this->productRepository->getById($productId, false, $storeId);
...
if ((int)$product->getStatus() !== Status::STATUS_ENABLED || !$product-
>isVisibleInSiteVisibility()) {
```

Loading a product **with a store id** is not cosmetic. `status` and `visibility` are store-view-scoped attributes, so a product that is Enabled on the UAE site can be Disabled on the India site. Loading without the store id would return the default-scope values and let a disabled product into a wish list. Pass the store id whenever you load a product outside a normal storefront request.

8.9 Customers belong to a website — `SeedCustomers.php`

`app/code/Codilar/Twoxl/Setup/Patch/Data/SeedCustomers.php`

```
private const SAMPLES = [
    ['email' => 'rahul.menon@example.com', ..., 'website' => 'in',   'store' => 'in_en'],
    ['email' => 'layla.m@example.com',     ..., 'website' => 'base', 'store' => 'ae_en'],
];
...
$customer->setWebsiteId($websiteId);   // ← which site they can log in to
$customer->setStoreId($storeId);       // ← which view they registered in (emails,
language)
```

Both are set, and they mean different things:

- `website_id` **is the account's identity.** The uniqueness check on email is per website — `$this->customerRepository->get($email, $websiteId)`. The same email address can be two completely separate accounts on two websites.
- `store_id` **is a preference.** It records where they signed up, so welcome emails and newsletters go out in the right language.

> **"Customer Account Sharing"** (`customer/account_share/scope`) flips this: set it to *Global* and one account works across all websites. Default is *Per Website*. Clients ask for Global surprisingly often — "why do I have to register twice?" — and it is a one-field change, but it means an email address can only exist once in the whole installation.

8.10 Store view context in templates — the theme layer

`app/design/frontend/Codilar/twoxl/Magento_Theme/layout/default.xml`

```
<block class="Magento\Framework\View\Element\Template" name="twoxl.region.strip"
       template="Magento_Theme::html/header/region-strip.phtml" after="twoxl.topbar">
    <arguments>
        <argument name="region_banner"
xsi:type="object">Codilar\Twoxl\ViewModel\RegionBanner</argument>
    </arguments>
</block>
```

The view model is injected as a block argument, so the template stays free of business logic:

```
/** @var \Codilar\Twoxl\ViewModel\RegionBanner $region */
$region = $block->getData('region_banner');

if (!$region->isEnabled()) {
    return;
}
```

That early `return` is the whole "turn the strip off for one region" feature. Same template, two outcomes, decided by one `core_config_data` row.

> **Full-page cache and store views.** Magento caches a separate FPC entry per store view, so per-store config differences are cached correctly and for free. What is *not* safe is anything per-**customer** — which is why `account.phtml` carries the comment *"Page is full-page cached, so the logged-in state is resolved client side from the customer section"*. Store view = safe to cache. Customer = must be a private-content section.

8.11 Root category through the store view — `Pdp.php`

`app/code/Codilar/Twoxl/ViewModel/Pdp.php`

```
$rootId = (int)$this->storeManager->getStore()->getRootCategoryId();
$ids = array_filter(
    array_map('intval', explode('/', (string)$primary->getPath())),
    static fn (int $id) => $id > 1 && $id !== $rootId
);
```

A neat demonstration of the hierarchy at runtime: you ask the **store view** for its root category, and it answers by delegating to its **store group** — because that is the level that actually owns the value. The breadcrumb then

strips the root (id 2) and the true root (id 1) out of the category path, so the shopper sees *Home → Sofas → Sectionals* rather than *Root Catalog → Default Category → Sofas*.

Because the code asks the store view rather than hard-coding `2`, this breadcrumb would keep working unchanged if you later gave India its own root category.

Part 9 — The traps you already hit (and what they teach)

Your codebase contains three real bugs that were found and fixed. They are better teachers than any textbook, because they are the exact mistakes everyone makes on their first multi-website build.

Trap 1 — The config cache made the price scope a lie

What happened. `ConfigureIndiaStore` wrote `catalog/price/scope = 1` (Website). Immediately afterwards, in the *same PHP process*, `SetIndiaCatalogPrices` wrote INR prices at store scope. But the price attribute had already been built from the **cached** config, where the scope was still Global. A global attribute ignores the store id you pass and writes to the default row — so the rupee prices landed on `store_id = 0` and **overwrote the dirham prices**. The UAE site started showing ₹102,999 as AED 102,999.

The fix, now the first line of `apply()` in

`app/code/Codilar/Twoxl/Setup/Patch/Data/SetIndiaCatalogPrices.php`:

```
$this->reinitableConfig->reinit();
```

with the explanation in the class docblock:

```
/**
 * The config is reinitialised first. ConfigureIndiaStore writes catalog/price/scope in
this same
 * process, and until the cached config is dropped the price attribute still reports global
scope -
 * a store scoped write would then collapse onto the default row and overwrite the AED
prices.
 */
```

And the clean-up patch, `app/code/Codilar/Twoxl/Setup/Patch/Data/RestoreRegionalPrices.php`, which puts both sets of numbers back where they belong:

```
$this->writePrices(self::AED_PRICES, 0);              // default row — UAE inherits it
$this->writePrices(self::INR_PRICES, $indiaStoreId);  // override — India only
```

> **The lesson.** `ScopeConfigInterface` is cached per request. Any code that *changes* configuration and then *depends* on that change in the same process must call `ReinitableConfigInterface::reinit()` in between. This bites hardest in data patches, setup scripts and console commands, where several unrelated things happen in one long-lived process.

Trap 2 — A new store view has no URL rewrites

What happened. The India store view was created after the catalogue already existed. Magento generates URL rewrites when a category or product is *saved* — so `url_rewrite` had rows for store 1 and none at all for store 2. Every category URL under `/in_en/` returned 404 while the homepage worked fine.

The fix, `app/code/Codilar/Twoxl/Setup/Patch/Data/GenerateRegionalUrlRewrites.php`:

```
$category = $this->categoryRepository->get((int)$item->getId(), 0);
$category->setStoreId(0);
// true = regenerate for every store the category reaches, not just the current one.
$this->persist($this->categoryUrlRewriteGenerator->generate($category, true));
```

Two details worth remembering:

- `generate($category, true)` — the second argument means "every store view this category reaches". Without it you would generate rewrites only for the store view the object was loaded in.
- The duplicate case is swallowed deliberately, because re-running the patch must be harmless:

  ```
  try {
      $this->urlPersist->replace($urls);
  } catch (\Magento\UrlRewrite\Model\Exception\UrlAlreadyExistsException) {
      // an equivalent rewrite is already in place
  }
  ```

> **The lesson.** URL rewrites are **per store view**. Whenever you add a store view to a live site, regenerate them — either with a patch like this one, or by re-saving the categories, or with `bin/magento indexer:reindex catalog_category_product catalog_product_category` followed by a category save. Expect this on every multi-store go-live.

Trap 3 — Area code and transactions in patches

Three of your patches implement `NonTransactionableInterface` and wrap their work in `emulateAreaCode()`:

```
class SetIndiaCatalogPrices implements DataPatchInterface, NonTransactionableInterface
...
$this->appState->emulateAreaCode(Area::AREA_ADMINHTML, function (): void { ... });
```

The docblock explains it:

```
/**
 * Non transactional because this updates existing products: Product::getIdentities() reads
the
 * area code, and PatchApplier defers commit callbacks past the emulateAreaCode() block.
 */
```

In plain terms: saving a product fires cache-invalidation callbacks. Those callbacks need to know which area (frontend/adminhtml) they are in. Inside a transaction, Magento defers them until commit — which happens *after* your `emulateAreaCode()` closure has already exited, so the area code is gone again and the save blows up with *"Area code is not set"*. Opting out of the transaction makes the callbacks run while the area is still emulated.

`SeedCustomers` hits the same thing and says so:

```
/**
 * Area code is emulated because saving a customer invalidates caches, and the tags are
built
 * from the current area; without one the save fails with "Area code is not set".
 */
```

Trap 4 — The one still waiting for you

Not a bug yet, but a documented assumption that will become one:

```
// India has a single view, so a store level write is the website value.
```

Prices are stored at **store view** scope for India. The day a second Indian store view is added (Hindi, say), it will inherit the default-scope AED prices, not the INR ones, because nothing was ever written at *website* scope. The correct long-term fix is to write prices once per website using `Magento\Catalog\Model\Product\Action::updateAttributes()` with each of that website's store ids, or to re-save the product with the website's default store id. Worth fixing before the client asks for Hindi, not after.

A checklist for the next multi-website project

1. Create website → group → store view, **in that order**; set the default pointers last.
2. Decide the root category: shared (one catalogue) or separate (two catalogues).
3. Set `catalog/price/scope` **before** touching any prices, then `reinit()`.
4. Set base currency at **website** scope; locale at **store view** scope.
5. Assign products to the new website (`updateWebsites(..., 'add')`) or the storefront will be empty.
6. Set base URLs at website scope, or turn on store codes in URLs — and rename the `default` view if you do.
7. Regenerate URL rewrites for the new store view.
8. Decide customer account sharing: per website or global.
9. Reindex and flush: `bin/magento indexer:reindex && bin/magento cache:flush`.
10. Check every custom table that stores user-generated data has a `store_id` column.

Part 10 — Cheat sheet, commands and exercises

PHP you will use constantly

```
use Magento\Store\Model\StoreManagerInterface;
use Magento\Store\Model\ScopeInterface;
use Magento\Framework\App\Config\ScopeConfigInterface;

// --- the current context ------------------------------------------------
$store   = $this->storeManager->getStore();          // the current STORE VIEW
$storeId = (int)$store->getId();                     // 1, 2, ...  (0 = admin)
$code    = $store->getCode();                        // 'ae_en'
$website = $this->storeManager->getWebsite();        // its website
$group   = $this->storeManager->getGroup();          // its store group
$rootId  = (int)$store->getRootCategoryId();         // from the group
$currency= $store->getCurrentCurrencyCode();         // display currency
$baseUrl = $store->getBaseUrl(UrlInterface::URL_TYPE_LINK);

// --- look one up by code ------------------------------------------------
$this->storeManager->getStore('in_en');
$this->storeManager->getWebsite('in');

// --- list them ----------------------------------------------------------
$this->storeManager->getStores();      // store views, admin excluded
$this->storeManager->getWebsites();
$website->getStoreIds();               // every store view under a website

// --- read config --------------------------------------------------------
$scopeConfig->getValue('twoxl/region/headline', ScopeInterface::SCOPE_STORE);
$scopeConfig->getValue('twoxl/region/headline', ScopeInterface::SCOPE_STORE, $storeId);
$scopeConfig->getValue('currency/options/base', ScopeInterface::SCOPE_WEBSITE, $websiteId);
$scopeConfig->isSetFlag('twoxl/region/enabled', ScopeInterface::SCOPE_STORE);

// --- write config -------------------------------------------------------
$writer->save($path, $value, ScopeConfigInterface::SCOPE_TYPE_DEFAULT, 0);
$writer->save($path, $value, ScopeInterface::SCOPE_WEBSITES, $websiteId);
$writer->save($path, $value, ScopeInterface::SCOPE_STORES,   $storeId);
$reinitableConfig->reinit();           // after writing, before depending on it
```

Two constants that look identical and are not. When **reading**, use `ScopeInterface::SCOPE_STORE` / `SCOPE_WEBSITE` (singular). When **writing** with the config writer, use `SCOPE_STORES` / `SCOPE_WEBSITES` (plural) — those are the literal values of the `scope` column. Mixing them up is a silent no-op that costs people an afternoon.

CLI commands

```
# apply your data patches (this is what creates the India website)
bin/magento setup:upgrade

# see what is registered
bin/magento store:list                  # not in core; use the admin, or:
bin/magento config:show                 # every config value, default scope
bin/magento config:show --scope=stores --scope-code=in_en
bin/magento config:show catalog/price/scope

# write config from the CLI
bin/magento config:set --scope=websites --scope-code=in currency/options/base INR
bin/magento config:set --scope=stores   --scope-code=in_en general/locale/code en_IN

# after anything scope-related
bin/magento indexer:reindex
bin/magento cache:flush
```

SQL for when you want to see the truth

```
SELECT w.website_id, w.code AS website, g.group_id, g.code AS store_group,
       g.root_category_id, s.store_id, s.code AS store_view, s.is_active
FROM   store_website w
JOIN   store_group   g ON g.website_id = w.website_id
JOIN   store         s ON s.group_id   = g.group_id
ORDER  BY w.website_id, g.group_id, s.store_id;

-- every override anyone has ever made
SELECT scope, scope_id, path, value
FROM   core_config_data
WHERE  scope <> 'default'
ORDER  BY scope, scope_id, path;

-- your own overrides
SELECT * FROM core_config_data WHERE path LIKE 'twoxl/%';

-- price rows per store view (attribute_id varies; find it in eav_attribute)
SELECT entity_id, store_id, value FROM catalog_product_entity_decimal
WHERE  attribute_id = (SELECT attribute_id FROM eav_attribute
                       WHERE attribute_code = 'price' AND entity_type_id = 4);
```

Exercises — do these on your own install

1. **See inheritance with your own eyes.** Run the SQL above for `twoxl/%`. Count the rows: four, all `scope='stores'`, all `scope_id=2`. There is nothing for the UAE — confirm the UAE strip still renders, and explain to yourself where its text came from.
2. **Break it deliberately.** In `RegionBanner::value()`, remove the `ScopeInterface::SCOPE_STORE` argument. Flush cache, load `/in_en/`. The Indian banner now shows the UAE wording. Put it back. You have now personally met the most common Magento config bug.

3. **Add a third region without writing code.** In the admin create website `uk` → group `uk_store` (root category 2) → store view `uk_en`. Set GBP at website scope and a banner headline at store view scope. Reindex, flush. The globe menu should now list three countries — because `RegionSwitcher` reads from the database, not from a list.
4. **Then find out what is missing.** Visit a category URL under `/uk_en/`. It will 404 until you regenerate URL rewrites. Fix it. You have now reproduced Trap 2 on purpose.
5. **Turn off a feature for one region.** Switch the config scope to *India English*, untick "Use Website" on *Enquiries → Product Enquiries → Enable Enquiry Form*, set it to No. Confirm the form disappears on `/in_en/` and survives on `/ae_en/`. Then POST to `twoxl/enquiry/save` from India anyway and confirm `Save.php` rejects it server-side too.
6. **Trace a price.** Pick SKU `127933-IVORY`. Find its two rows in `catalog_product_entity_decimal` (store 0 and store 2). Change the store-2 value directly in SQL, flush cache, reload the India PDP. Then put it back with a patch rather than SQL — the way `RestoreRegionalPrices` does it.
7. **Explain the hierarchy out loud** using your own files: which file creates each level, which scope each config write targets, and why `catalog/price/scope` had to be global. If you can do that without notes, you know this topic.

One-page summary

Question	Answer
Different currency?	Website
Separate customer logins?	Website
Separate cart / checkout?	Website
Different payment or shipping methods?	Website
Different product range / menu?	Store Group (root category)
Different language or translations?	Store View
Different theme or design?	Store View
Different CMS pages / blocks?	Store View
Different product name or description?	Store View
Product enabled here, disabled there?	Store View (`status` , `visibility`)
Different price for the same product?	Website — after setting Catalog Price Scope = Website
Product sold here, not sold there?	Website assignment (`updateWebsites`)
Different banner text / colour?	Store View config (your `twoxl/region/*`)
Store codes in URLs, cron, indexers, SKU?	Default / Global — cannot vary

> **If you remember nothing else: Website = money and customers. Store Group = products. Store View = words.**
> Everything in `Codilar_Twoxl` is one of those three sentences turned into code.